

PILARES DO DESENVOLVIMENTO

Detalhamento sobre o que a turma precisa entregar para que todas as sete equipes possam trabalhar em seus módulos de forma autônoma e eficiente.

Considerando o uso de **.NET MAUI**, a **conexão direta ao banco de dados** e a divisão em **módulos** por tema, as dependências comuns são bem específicas e se concentram na fundação do aplicativo.

Tarefas e Entregas das Dependências em comum

A sala inteira atuará como uma equipe de "fundação" ou "infraestrutura", será responsável por criar e manter a arquitetura central do aplicativo. Suas principais tarefas são:

1. Módulo de Autenticação e Autorização

- **Login com SSO:** A equipe de fundação deve implementar a lógica de autenticação via SSO (Single Sign-On). Isso inclui a interface de login, a comunicação com o provedor de identidade da instituição e o gerenciamento da sessão do usuário.
- **Controle de Acesso:** Criar um módulo de autorização que, após o login, identifique o papel e o escopo do usuário (por exemplo, quais módulos ele pode ver). Essa lógica precisa ser acessível a todos os outros times.
- **Gerenciamento de Credenciais:** Otimizar o armazenamento seguro das credenciais e tokens de acesso no dispositivo (por exemplo, usando o *SecureStorage* do .NET MAUI).

2. Módulo de Conexão com o Banco de Dados

- **Camada de Conexão:** Desenvolver uma **camada de abstração** de banco de dados. Apesar da conexão ser direta, não é recomendado que cada time crie sua própria conexão. A equipe de fundação deve criar uma classe ou serviço que centralize as strings de conexão, gerencie a abertura e o fechamento de conexões e lide com possíveis erros. Isso garante que todos os times usem a mesma conexão, evita a duplicação de código e simplifica a manutenção.
- **Repositório de Dados Genérico:** Criar um padrão de acesso a dados (como um padrão **Repository**) para que os outros times possam interagir com o banco de forma padronizada. Por exemplo, uma classe base que implemente métodos de consulta, inserção ou atualização genéricos.
- **Lógica de Consulta:** Estabelecer as convenções para as consultas SQL. Se os times farão consultas diretas, a equipe de fundação pode oferecer exemplos ou funções auxiliares para evitar erros comuns de segurança e performance.

3. Componentes de UI/UX Comuns

- **Design System:** O principal aqui é criar uma **biblioteca de componentes reutilizáveis**. No .NET MAUI, isso pode ser feito com **Custom Controls** ou **Styles** que definem a aparência de botões, inputs, cards e listas. Essa biblioteca

deve garantir que a identidade visual do aplicativo (cores, fontes, tamanhos) seja consistente em todos os módulos.

- **Templates de Páginas:** Fornecer um "esqueleto" de páginas (Page Templates) com a navegação padrão (barra de menu inferior, barra superior) para que cada time possa apenas preencher com o conteúdo de seu módulo.
- **Padrão de Navegação:** Definir e documentar o fluxo de navegação entre as telas dos diferentes módulos. Por exemplo, como uma tela do "Módulo Alunos" navega para uma tela de "Detalhes do Aluno" que pode estar em outro módulo.

4. Outras Tarefas Comuns

- **Tratamento de Erros:** Definir um padrão global para o tratamento de erros. A oitava equipe deve criar um sistema de log ou de notificação de erros que centralize o registro de falhas, sejam elas de conexão, de dados ou de interface.
- **Central de Mensagens/Notificações:** Criar um serviço de mensagens ou "event bus" que permita que os módulos se comuniquem entre si de forma desacoplada. Por exemplo, se o Módulo de *Disciplinas* altera um dado, ele pode notificar o Módulo de *Cronograma* sobre a mudança.
- **Serviços de Cache:** Implementar um serviço de cache para que dados frequentemente acessados, como listas de opções ou dados de perfil do usuário, não precisem ser consultados no banco a todo momento. Isso melhora o desempenho e a experiência do usuário.
- **Módulo de Offline:** Como mencionado no escopo, o suporte offline é limitado. A equipe de fundação pode ser responsável por criar a lógica de caching para os dados de dashboards e telas principais, permitindo uma consulta básica sem conexão.

Em resumo, a sala não construirá um módulo funcional completo, mas sim os **pilares** sobre os quais todos os outros sete módulos serão construídos. Eles são responsáveis por garantir que as tarefas complexas e repetitivas, como segurança, conexão com dados e consistência visual, sejam feitas uma única vez e de forma robusta, liberando os outros times para se concentrarem em suas lógicas de negócio e visualizações específicas.

DESIGN

Um aplicativo com sete botões gigantes na tela inicial ficaria com uma aparência "crua" e não seria intuitivo. A chave é ir além da funcionalidade e focar na **experiência do usuário (UX)** e na **identidade visual (UI)**.

Vamos abordar essa questão em duas frentes: a definição de padrões para os dashboards e a pesquisa de estilos para garantir que a UI se alinhe à identidade da instituição de ensino.

1. Definindo Padrões de Dashboards por Módulo

Em vez de cada equipe criar seus dashboards do zero, o time de fundação deverá fornecer um conjunto de **templates e componentes** para guiar a criação. Isso garante consistência e usabilidade em todo o aplicativo.

- **Padrão de Layout:**
 - **Tela de Resumo:** A página principal de cada módulo pode ter um layout de "cards", onde cada card representa um KPI ou um resumo de dados importantes. Por exemplo, o Módulo de Alunos poderia ter cards de "Alunos Ativos", "Calouros", "Evasão", etc.
 - **Navegação Interna:** Defina uma forma padrão para navegar dentro do módulo. Por exemplo, uma barra de navegação superior (top-nav) ou abas para alternar entre "Dashboard", "Listas Detalhadas" e "Formulários".
 - **Filtros:** A posição dos filtros (chips, dropdowns) deve ser padronizada (por exemplo, no topo da tela). Isso faz com que os usuários saibam onde encontrar as opções de segmentação, independentemente do módulo que estiverem usando.
- **Templates de Visualização:**
 - **Tipos de Gráficos:** O time de fundação pode criar componentes reutilizáveis para os tipos de gráficos mais comuns, como barras, linhas e donuts, já com cores e estilos padronizados.
 - **Tabelas:** Padronize a aparência de todas as tabelas, incluindo headers, ordenação e paginação.
 - **Card de KPI:** Crie um componente de card que exiba a métrica principal, sua tendência (ícone de seta para cima/baixo) e um rótulo claro.
- **Incentivo à Criatividade:**
 - **Workshops de Design Sprint:** A equipe de fundação pode realizar workshops curtos para fazer um brainstorming em conjunto, desenhando wireframes (rascunhos de telas) para seus módulos. Isso ajuda a visualizar a experiência de cada módulo antes de escrever qualquer linha de código.
 - **Protótipos Navegáveis:** A equipe de fundação também pode usar ferramentas como o Figma para criar um protótipo navegável simples. Isso permite que os times testem a fluidez da navegação e visualizem como a aplicação final se comportará.

2. Pesquisa de Estilos para a Identidade Visual

Para que a interface se adeque à identidade da instituição de ensino, a sala deve realizar uma pesquisa e documentação visual. As informações a serem buscadas são:

- **Cores:** A primeira e mais importante etapa é descobrir a **paleta de cores oficial** da instituição. Isso inclui a cor principal, cores secundárias e cores complementares. Geralmente, as instituições de ensino superior possuem um manual de identidade visual. Os alunos podem buscar por "manual de marca", "guia de identidade" ou "brand book" no site da instituição.
- **Fontes (Tipografia):** Qual a fonte oficial usada em logos, materiais de divulgação, site e documentos? A oitava equipe deve identificar se é uma fonte gratuita ou licenciada e, caso seja licenciada, buscar uma fonte gratuita similar para o aplicativo. A tipografia transmite a personalidade da marca, seja ela moderna, tradicional, etc.
- **Logotipo e Ícones:** A equipe precisa do arquivo de alta resolução do logotipo oficial. Além disso, é importante procurar por um estilo de ícones que se harmonize com o logo (ex: ícones com cantos arredondados, preenchidos, lineares). A própria instituição pode ter uma biblioteca de ícones.
- **Recursos Visuais:**
 - **Estilo Fotográfico:** A instituição usa fotos com pessoas em ambientes de estudo, fotos conceituais, fotos mais profissionais? Entender o estilo das imagens ajuda a criar uma atmosfera visual coesa.
 - **Formas e Grafismos:** A marca usa elementos visuais específicos, como bordas arredondadas, formas geométricas ou linhas em suas artes? A incorporação desses elementos pode reforçar a identidade.

Uma vez que essas informações sejam coletadas, a sala deve criar um **Guia de Estilo (Style Guide)** simples. Este guia servirá como a "bíblia" visual do projeto, contendo as paletas de cores, fontes, exemplos de botões, cards, e os ícones a serem usados.

Ao fazer isso, a sala capacita os todas as equipes separadamente a construir módulos que, mesmo sendo independentes, parecem fazer parte de uma mesma aplicação coesa e bem projetada, elevando a qualidade do projeto como um todo.

GERENCIAMENTO DO PROJETO

o GitHub é uma ótima opção para este projeto. A integração nativa com o Visual Studio e a Microsoft é um ponto forte. O GitHub é a plataforma de versionamento mais popular do mundo e a mais usada pela comunidade de desenvolvedores. Com um time de 40 alunos, o GitHub oferece:

- **Ferramentas de Colaboração:** Pull Requests com revisões de código, comentários, e a possibilidade de visualizar as mudanças antes de serem integradas.
- **Gestão de Projetos:** Quadros Kanban ou Scrum (via GitHub Projects) para que os times possam organizar suas tarefas, atribuir issues e acompanhar o progresso.
- **Comunidade:** A maioria dos alunos provavelmente já conhece ou precisará aprender a usar o GitHub, o que faz com que a curva de aprendizado seja menor.

Táticas de Versionamento para o Projeto

Com 7 equipes e o time de infraestrutura, a escolha da estratégia de versionamento é crucial para evitar conflitos e manter a organização. A ideia é realizar um Pull Request semanal para cada equipe.

Trunk-Based Development

Como um modelo mais simples, o Trunk-Based Development é uma boa alternativa.

- **main branch:** É a única branch de longa duração.
- **feature branches:** As equipes trabalham em branches curtas e fazem merges frequentes na main branch.

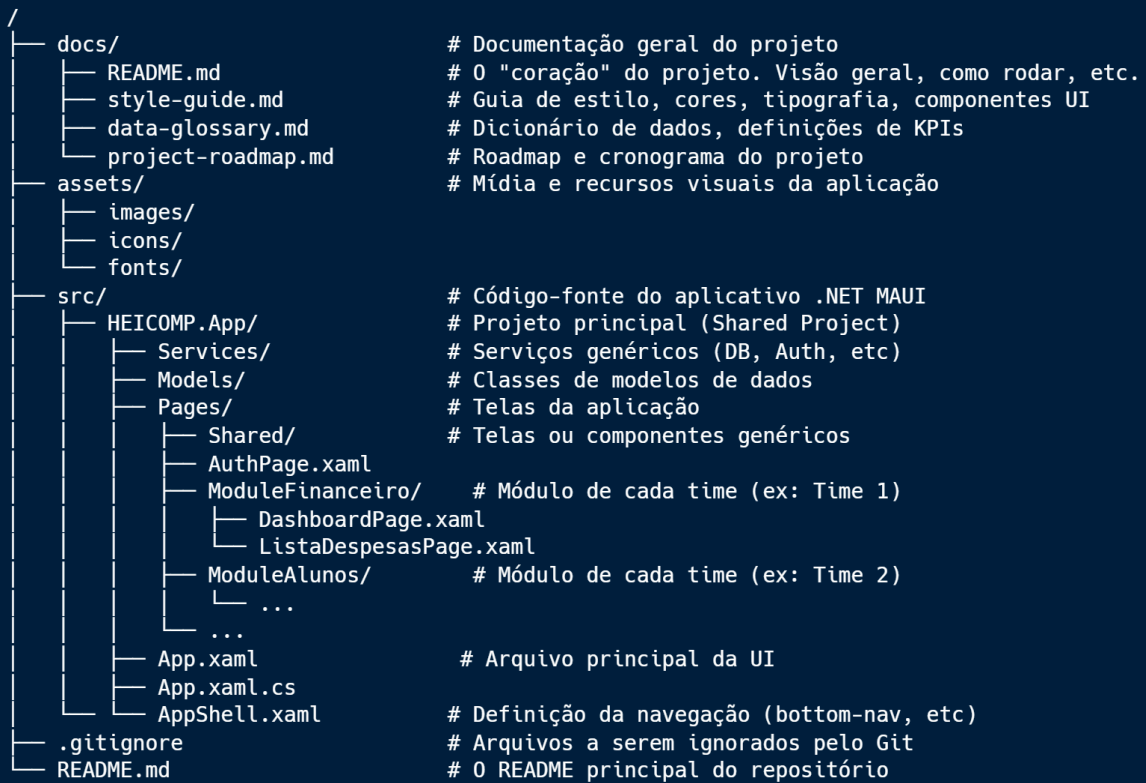
Vantagens:

- Menos complexidade, com um único branch principal.
- Incentiva a integração contínua e a entrega de pequenas partes de código.
- A curva de aprendizado é menor para quem não tem experiência com Git.

Metodologia de versionamento no repositório:

1. **Criação de branches:** Cada uma das 7 equipes de alunos cria uma **única branch semanal** para seu trabalho. ex.: `feature/modulo-financeiro-semana-1`.
2. **Desenvolvimento:** Durante a semana, todos os membros da equipe contribuem para essa mesma branch.
3. **Pull Request Semanal:** No final da semana, a equipe designa um membro para abrir um Pull Request da `feature/modulo-financeiro-semana-1` para a `main`.
4. **Revisão:** A equipe de infraestrutura ou outro aluno pode ser designado para fazer a revisão do código (se o escopo do projeto permitir).
5. **Merge:** Após a aprovação, o Pull Request é integrado à branch `main`.

Essa abordagem garante o controle de qualidade, a visibilidade do trabalho e a prática de um fluxo de trabalho colaborativo, preparando os alunos para o mercado de trabalho.



ROADMAP DO PROJETO

1. **Descoberta:** Entrevistas, priorização de KPIs, mapeamento de fontes.
2. **Fundacional:** Autenticação/Autorização, Responsividade Mobile-First, Ingestão de Dados Mínima, Primeiros Dashboards.
3. **Valor Incremental 1 :** Drill-Down & Exportação, Alertas & Metas, Catálogo de Dados, Auditoria Básica.
4. **Valor Incremental 2 :** Detalhamento de Alertas & Metas, Acessibilidade, Auditoria Detalhada.
5. **Polimento:** Otimizações de Performance, Testes de Usabilidade.

A ideia por trás dos **Valores Incrementais** é que o projeto seja construído em etapas, entregando funcionalidades que agregam valor ao usuário final, mesmo antes de o produto estar 100% completo. Em vez de entregar tudo de uma vez, iremos entregar valor de forma contínua, permitindo feedback e validação do cliente (a instituição de ensino).

Vamos detalhar cada um dos incrementos.

Valor Incremental 1: Funcionalidades Essenciais

Nesta fase, a equipe se concentra em construir a base das funcionalidades interativas e de governança. O foco é transformar os dados estáticos em algo que o usuário pode explorar e com o qual pode interagir.

- **Drill-Down e Exportação:** Os dashboards deixam de ser apenas gráficos bonitos. O usuário agora pode clicar em um ponto do gráfico ou em um card de KPI e aprofundar a análise, vendo os dados mais detalhados. Por exemplo, ao clicar em "Matrículas Ativas", ele vê a lista de alunos com essa situação. Além disso, a funcionalidade de exportação (CSV e PNG) permite que o usuário leve os dados para fora da aplicação, tornando o sistema uma ferramenta de trabalho mais completa.
- **Alertas & Metas (Básico):** Os dashboards ganham inteligência. É possível definir uma meta simples para um KPI (por exemplo, "Taxa de Evasão não deve ser maior que 10%"). Se o valor real ultrapassar a meta, o aplicativo exibe um alerta visual (por exemplo, um badge ou uma mudança de cor). Isso permite que o gestor se concentre nos indicadores que realmente precisam de atenção.
- **Catálogo de Dados:** A transparência é fundamental. Este item não é sobre funcionalidade, mas sobre **confiança**. O aplicativo passa a ter uma página (ou uma seção) que descreve o que cada KPI significa, qual sua fórmula e de onde o dado veio. Isso evita que diferentes pessoas usem as mesmas métricas com definições diferentes, garantindo consistência e credibilidade.
- **Auditoria Básica:** Uma funcionalidade essencial para segurança e rastreabilidade. Implementa-se um registro simples de eventos, como a publicação ou alteração de um dashboard. Isso permite que o admin saiba "quem fez o quê e quando", o que é um primeiro passo para a conformidade e a segurança dos dados.

Valor Incremental 2: Polimento e Aprofundamento

Esta fase é dedicada a aprimorar as funcionalidades já existentes e a adicionar camadas de robustez e inclusão. O foco aqui é melhorar a **usabilidade**, a **acessibilidade** e a **confiabilidade**.

- **Detalhamento de Alertas & Metas:** O sistema de alertas evolui. A capacidade de definir metas e alertas se torna mais granular. Em vez de um alerta visual simples, ele pode ser configurado para enviar uma notificação por e-mail, por exemplo. Isso torna o sistema proativo, informando o gestor sobre um problema antes que ele precise abrir o aplicativo.
- **Acessibilidade:** Uma etapa de extrema importância. O aplicativo é otimizado para ser acessível a todos os usuários, incluindo aqueles com deficiências. Isso envolve implementar requisitos como contraste de cores suficiente, navegação por teclado e compatibilidade com leitores de tela (labels alternativos). Esta etapa demonstra o compromisso do projeto com a inclusão.
- **Auditoria Detalhada:** A auditoria básica evolui. O sistema agora registra eventos de forma mais detalhada, incluindo histórico de execuções de carga de dados. Isso significa que, se houver um problema com a qualidade dos dados em um determinado dia, o administrador pode rastrear a origem e a causa do erro, o que é vital para a manutenção e a confiabilidade do sistema.

Em resumo, o **Valor Incremental 1** foca no '**o que**' (as funcionalidades-chave), enquanto o **Valor Incremental 2** foca no '**como**' (a qualidade e a forma como as funcionalidades são entregues e aprimoradas). Essa abordagem passo a passo garante que o projeto entregue valor de forma contínua e que a versão final seja robusta, confiável e inclusiva.

REFERÊNCIA:

<https://chatapi.unis.edu.br/heicomp.html>